TECHNICAL WHITEPAPER

SATS-402

Bitcoin-Native Atomic Delivery for AI-Agent Commerce

A technical whitepaper for preimage-locked API, MCP, session, and streaming settlement over Lightning.

VERSION
v0.6 technical diligence draft

AUTHOR
Maven Jang, Founder, SATS-402

STATUS
Protocol design + working local real-LND proof

DATE
April 2026

Document Status and Scope

This paper specifies SATS-402 as a Bitcoin-native protocol family for paid delivery of API responses, MCP tool results, long-running sessions, and streaming data used by AI agents. It is written for technical diligence and strategic evaluation, not as legal, tax, or financial advice.

The current implementation evidence consists of a local regtest proof using real `bitcoind`, three real LND nodes, active Lightning channels, real hold invoices, real merchant payments, real preimage observation, same-hash settlement, CLTV safety checks, and local response decryption. Public market proof includes permissionless Firecrawl-style and Apify-style wrappers that demonstrate the same paid-delivery primitive against AI infrastructure surfaces.

> **Core claim**
>
> SATS-402 does not try to make every AI-agent payment a custodial balance, a card account, a stablecoin account, or a generic payment button. It uses Lightning payment preimages as cryptographic delivery roots for paid digital resources, while keeping the facilitator out of custody and out of balance-sheet credit exposure.

Abstract

AI agents are becoming economic actors. They call APIs, buy web data, rent compute, trigger automation tools, and interact with MCP servers. The emerging standards around HTTP 402 payments, agent authorization, and machine-to-machine commerce are creating a new settlement layer for software. Without a Bitcoin-native path, that layer is likely to default to EVM rails, stablecoin processors, or closed fintech infrastructure.

SATS-402 is a protocol architecture for Bitcoin-native paid delivery. Its first primitive, **Atomic Response**, encrypts a paid API or MCP response before delivery and derives the local decryption key from a real Lightning payment preimage and an agent-merchant ECDH secret. Its second primitive, **Same-Hash HTLC Bridge**, allows a facilitator to coordinate an incoming hold invoice and outgoing merchant payment under the same payment hash, enforcing an asymmetric CLTV safety invariant so the facilitator does not become a credit institution. Its third and fourth primitives, **Session Ticket** and **Chunked Streaming Settlement**, extend the design to repeated calls, long-running sessions, and streams without opening one HTLC per data chunk.

SATS-402 composes with x402, L402, and AP2 rather than replacing them. x402 provides the HTTP 402 payment negotiation and facilitator pattern. L402 provides the conceptual model for Lightning preimage-backed access credentials. AP2 provides the authorization and accountability layer through signed mandates. SATS-402 adds the missing Bitcoin-native delivery mechanism: payment is not merely a receipt; it becomes the cryptographic root that unlocks the purchased digital resource.

Table of Contents

1. The Problem: Agents Need Paid Delivery, Not Human Checkout

The web payment stack was built around humans: accounts, cards, checkout pages, billing relationships, passwords, support desks, and fraud models. AI agents do not fit that pattern. An autonomous or semi-autonomous agent wants to call an API, buy a bounded unit of data, rent a short compute task, run a browser automation tool, or execute an MCP tool without creating a human-centric billing relationship for every call.

The first economic unit of agentic commerce is not a subscription. It is a paid request.

1.1 Why existing payment flows are insufficient

- **Small value:** fractions of a cent to cents per call.
- **High frequency:** repeated tool calls and repeated data retrieval.
- **Programmatic:** no human present during execution.
- **Budgeted:** the agent needs a spending limit and policy constraints.
- **Delivery-sensitive:** the resource should be returned only when the payment condition is satisfied.
- **Auditable:** merchants, agents, and users need proof of what was authorized, paid, delivered, and consumed.

HTTP 402, x402, L402, and AP2 all point toward the same macro-shift: software agents will need native commerce rails. SATS-402 asks one narrower question: *if that commerce layer forms, why should Bitcoin be absent from its settlement and delivery primitive?*

1.2 The cold limit of a generic Bitcoin payment adapter

A simple Lightning payment SDK is not enough. If a server says "pay this invoice, then I will return data," the server still has to be trusted to return the correct data after payment. If a facilitator guarantees delivery when routing fails, it starts absorbing credit and balance-sheet exposure. If every streamed chunk opens a separate HTLC, Lightning channel limits, route latency, and HTLC slot pressure make the design fragile.

SATS-402 is therefore not a generic payment adapter. It is a delivery protocol that uses payment preimages, CLTV constraints, session tickets, and epoch-level settlement to bind payment, authorization, and resource access.

2. Related Protocols and Why SATS-402 Composes With Them

2.1 x402: HTTP 402 payment negotiation and facilitator pattern

x402 defines a practical pattern for paid HTTP resources. A resource server can respond with `402 Payment Required`; a client can produce a payment payload; and a facilitator can verify and settle that payment so the server does not need to maintain direct chain connectivity or implement payment verification itself. The x402 documentation describes the facilitator as optional but recommended, and states that it performs verification and execution without acting as a custodian.[1]

SATS-402 adopts this facilitator pattern but changes the settlement and delivery primitive. In x402, the normal flow is: pay, verify, settle, then server returns the resource. In SATS-402, the response may already be encrypted and returned with the payment challenge; the Lightning preimage itself becomes part of the decryption key.

2.2 L402: Lightning preimage-backed access credentials

L402 uses a token, typically a Macaroon, tied to a Lightning payment hash. The token becomes valid only when presented with the corresponding Lightning preimage. Lightning Labs describes L402s as Macaroons that include a payment hash and require the preimage obtained by paying a Lightning invoice.[3,4]

SATS-402 borrows this insight but applies it differently. L402 is primarily an authorization credential for API access. SATS-402 uses the preimage as a delivery root: the paid response remains encrypted unless the agent obtains the preimage.

2.3 AP2: mandates and accountability for agent-led payments

Google's Agent Payments Protocol introduces a mandate-based model for agent payments. AP2 uses tamper-proof, cryptographically signed mandates to prove user instructions, capture authorization, and provide accountability when agents transact. It is designed as a payment-agnostic framework that can work with A2A, MCP, cards, stablecoins, and other payment methods.[5]

SATS-402 treats AP2-style mandates as the authorization layer. A mandate says what an agent may buy, from whom, under which constraints, and within which budget. SATS-402 then provides the Bitcoin-native paid-delivery layer that enforces resource unlock by preimage.

2.4 Why SATS-402 is not redundant

Protocol	Primary role	What SATS-402 adds
x402	HTTP payment negotiation and facilitator verification/settlement.	Bitcoin/Lightning-specific preimage-locked delivery and CLTV-safe same-hash bridging.
L402	Lightning preimage-backed API credential.	Response encryption, same-hash bridge, delivery receipts, session/streaming settlement.
AP2	Agent authorization, mandates, audit trail, accountability.	Lightning settlement and cryptographic resource delivery under that authorization.
MCP	Tool discovery and invocation for agents.	Paid delivery and settlement for MCP tool results.

3. SATS-402 Design Thesis

Design decision	Protocol implication
Payment is delivery, not just receipt.	A Lightning payment preimage becomes an input to resource decryption.
The preimage is never used raw.	The content key is derived from preimage plus ECDH, so the gateway cannot decrypt.
The facilitator must not become a credit institution.	Core bridge settles agent-side hold invoice only after merchant-side settlement succeeds.
Long sessions need tickets.	Session Ticket mode encodes budget, expiry, caveats, and agent binding.
Streaming needs epoch-level payment roots.	Chunk keys are derived locally from epoch preimages instead of one HTLC per chunk.

```
H = SHA256(S)
K_response = HKDF(S || ECDH(agent, merchant), salt, info)
```

Streaming thesis

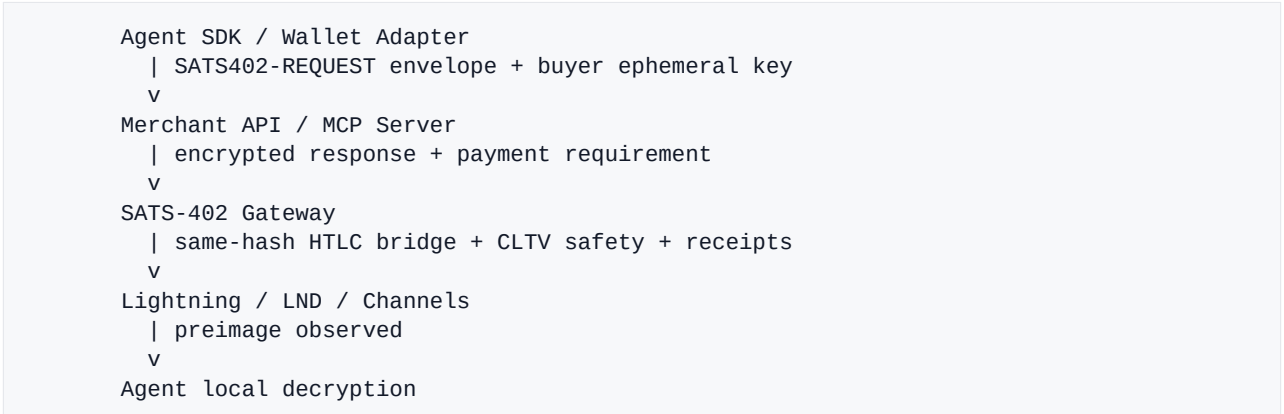
SATS-402 does not open one HTLC per byte or per chunk. It opens one Lightning payment per settlement epoch and derives per-chunk keys locally from the epoch preimage.

4. System Architecture

4.1 Actors

Actor	Role
Agent	The buyer. It requests API/MCP resources, holds a wallet or wallet adapter, and receives encrypted responses.
Merchant	The seller. It produces API responses, MCP tool results, Actor outputs, browser-session chunks, or streaming data.
SATS-402 Gateway	The facilitator. It coordinates payment negotiation, same-hash hold invoice bridging, CLTV safety, receipts, and optional session/stream policy.
Lightning nodes	LND/CLN/LDK or provider-backed nodes used by agents, gateway, or merchants. The reference proof uses LND.
Policy engine	Optional component that enforces AP2-style mandates, budgets, tool allowlists, and approval thresholds.

4.2 Logical architecture



5. Protocol Mode 1: Atomic Response

Atomic Response is the first and narrowest SATS-402 primitive. It is designed for discrete paid resources: a scrape result, an API snapshot, an MCP tool result, a small inference output, a database lookup, or an Apify Actor result after completion.

5.1 Flow

- Agent sends a SATS-402-aware request containing a request envelope and ephemeral public key.
- Merchant computes or retrieves the response.
- Merchant generates preimage S, payment hash H = SHA256(S), and merchant ephemeral key.
- Merchant encrypts the response using a key derived from S and ECDH.
- Merchant or Gateway issues payment requirements tied to H.
- Agent pays over Lightning.
- Agent receives or observes preimage S.
- Agent derives the response key locally and decrypts the ciphertext.
- Receipt is issued with payment, delivery, and authorization evidence.

5.2 Request envelope

```
{
  "v": "0.4",
  "mode": ["atomic_response", "atomic_bridge"],
  "buyer_eph_pk": "base64url...",
  "buyer_nonce": "base64url...",
  "payment_id": "pay_...",
  "request_hash": "sha256(method + url + body_hash)",
  "max_price_usd": "0.01",
  "wallet_caps": { "lightning": true, "preimage_return": true, "hold_invoice": true },
  "accept": {
    "encryption": ["xchacha20poly1305"],
    "kdf": ["hkdf-sha256"],
    "delivery": ["ciphertext-in-402", "paid-retry"]
  }
}
```

5.3 Idempotency

Retries must not create duplicate charges or inconsistent response keys. SATS-402 uses `payment_id` and `request_hash` to bind a logical request to its cached encrypted response. This mirrors the x402 payment-identifier extension, which provides a mechanism for clients to retry without duplicate payment processing by caching responses keyed by payment identifiers.[2]

6. Protocol Mode 2: Same-Hash HTLC Bridge

Atomic Response can be direct. In practice, agents and merchants often do not have direct channels or routing reliability. The gateway must bridge without extending credit.

6.1 Same-hash construction

```
H_agent_gateway = H_gateway_merchant = SHA256(S)
```

- Merchant prepares encrypted payload and invoice tied to H.
- Gateway creates an agent-facing hold invoice tied to the same H.
- Agent pays the gateway hold invoice; the invoice is accepted but not settled.
- Gateway pays the merchant invoice.

- Merchant settlement reveals S.
- Gateway uses S to settle the agent-side hold invoice.
- Agent obtains S and decrypts locally.

LND supports the required primitives: AddHoldInvoice creates a hold invoice tied to a supplied hash, and SettleInvoice settles an accepted invoice with a preimage.[6,8]

6.2 CLTV safety invariant

The same-hash bridge is unsafe without asymmetric time-lock control. The gateway must have enough time to use the merchant-side preimage to settle the agent-side hold invoice.

```
E_in >= E_out + Delta_safe
```

Where E_in is the expiry height of the incoming Agent-to-Gateway HTLC, E_out is the first-hop expiry height of the outgoing Gateway-to-Merchant HTLC, and Delta_safe is the bridge safety margin in blocks.

6.3 Gateway does not bridge blind

- merchant invoice payment hash equals expected H;
- amount and currency quote match the declared requirements;
- outgoing route total time lock is known;
- E_in >= E_out + Delta_safe;
- merchant final CLTV delta and route total CLTV are within policy;
- merchant in-flight HTLC quota is not exceeded;
- system HTLC pressure is below threshold.

Core safety posture

SATS-402 does not claim raw Lightning routing is always reliable. It assumes the opposite and turns routing, CLTV, liquidity, receipts, and preimage delivery into the product surface.

7. Protocol Mode 3: Session Ticket

Atomic Response is too narrow for repeated calls and long-running sessions. SATS-402 Session introduces a ticket that grants bounded access under explicit caveats.

7.1 Session ticket vs stored value

The ticket is not a user balance. It is a service access artifact that encodes authorization, budget, resource constraints, expiry, and receipt requirements.

```
{
  "ticket_type": "sats402_session_ticket",
  "stream_id": "strm_abc",
  "agent_id": "research-agent-01",
  "merchant_id": "browserbase-wrapper-01",
  "allowed_resources": ["web-data", "browser-session", "actor-results"],
  "budget": { "max_total_usd": "1.00", "remaining_usd": "0.72", "max_epoch_usd": "0.01" },
  "caveats": {
    "expires_at": "2026-04-26T12:00:00Z",
    "max_duration_seconds": 3600,
    "max_unpaid_chunks": 0,
    "bind_to_agent_pubkey": true,
    "bind_to_resource_hash": true
  }
}
```

The design intentionally resembles L402 in its use of preimage-backed access, but adds budget accounting, delivery receipts, response locking, and streaming constraints.

8. Protocol Mode 4: Chunked Streaming Settlement

8.1 The cold truth

SATS-402 Atomic cannot efficiently dominate every streaming case. A browser session lasting an hour or a financial tick stream emitting thousands of events per minute cannot open one Lightning HTLC per chunk. Doing so would create HTLC pressure, route failures, latency spikes, and channel liquidity issues.

> **Design constraint**
>
> SATS-402 Stream must never be implemented as one Lightning HTLC per data chunk. Lightning payments should occur at settlement epochs. Chunk keys must be derived locally from epoch preimages.

8.2 Epoch-level settlement

```
Epoch 0 invoice -> preimage P0 -> unlock chunks 0..99
Epoch 1 invoice -> preimage P1 -> unlock chunks 100..199
Epoch 2 invoice -> preimage P2 -> unlock chunks 200..299
```

A Lightning preimage becomes an epoch root. Per-chunk keys are derived locally.

```
K_epoch,e = HKDF(P_e || ECDH, stream_id || epoch_id, "sats402-stream-epoch-v1")
```

```
K_chunk,i = HKDF(K_epoch,e, stream_id || epoch_id || seq_i, "sats402-stream-chunk-v1")
```

8.3 Stream request

```
{
  "v": "0.5-stream",
  "mode": "chunked_streaming_settlement",
  "agent_id": "research-agent-01",
  "buyer_eph_pk": "base64...",
  "request_hash": "sha256(method+url+body)",
  "stream_id": "strm_...",
  "accepted_transports": ["sse", "websocket", "grpc"],
```

```
  "budget": { "max_total_usd": "1.00", "max_epoch_usd": "0.01", "max_per_second_usd": "0.05" },
  "policy": { "max_duration_seconds": 3600, "max_unpaid_chunks": 0, "resume": true, "requires_receipts": true },
  "ap2_mandate_hash": "optional_hash"
}
```

8.4 Long-running compute rule

For long-running Apify Actors, Browserbase sessions, crawling jobs, or GPU inference, SATS-402 must not hold a Lightning HTLC open for the entire compute period. The safe default is: complete the compute or milestone first, prepare the result, initiate SATS-402 paid delivery, then encrypt, settle, observe preimage, and decrypt locally.

> **Production rule**
>
> SATS-402 does not finance long-running compute with an open HTLC. It settles prepared results, bounded milestones, or pre-priced streaming epochs.

9. Cryptographic Model

9.1 Response-locked encryption

```
H = SHA256(S)
shared = ECDH(sk_agent, pk_merchant)
K = HKDF(S || shared, salt, info)
C = AEAD_Encrypt(K, nonce, plaintext, AAD)
```

Associated data should include the request hash, payment id, merchant id, resource hash, declared price, protocol version, and delivery mode.

9.2 Why gateway observation of the preimage is not enough

The gateway may observe S when settling the bridge. However, without shared = ECDH(sk_agent, pk_merchant), the gateway cannot derive the response key. This preserves content confidentiality from the facilitator.

9.3 Merkle commitments for streams

Streaming receipts commit to chunk ciphertext using a Merkle root. A dispute can demonstrate which encrypted chunks were delivered and under which epoch payment.

10. Security Invariants

ID	Invariant	Purpose
I1	Merchant and gateway invoices use the same payment hash in bridge mode.	Prevents mismatch between agent-side and merchant-side settlement.
I2	E_in >= E_out + Delta_safe.	Protects gateway from CLTV griefing and late preimage reveal.
I3	Agent-side hold invoice is settled only after outgoing merchant payment succeeds.	Prevents facilitator credit exposure in core mode.
I4	Gateway does not hold the ECDH content secret.	Prevents facilitator from decrypting responses.
I5	Payment ID and request hash bind retries to a single logical request.	Prevents duplicate billing and replay confusion.
I6	Streaming uses one Lightning payment per epoch, not per chunk.	Prevents HTLC slot exhaustion and routing fragility.

ID	Invariant	Purpose
I7	Session tickets are bound to agent public keys, resource hashes, expiry, and budget.	Reduces stolen-ticket and replay risk.
I8	Cryptographic delivery does not claim semantic correctness.	Separates delivery proof from quality disputes.

11. Threat Model

11.1 Adversaries

- **Malicious agent:** attempts replay, underpayment, old ticket reuse, or unauthorized resource access.
- **Malicious merchant:** attempts late preimage reveal, garbage payload delivery, excessive CLTV settings, or misleading pricing.
- **Malicious or compromised gateway:** attempts content inspection, incorrect receipt issuance, or unauthorized settlement state changes.
- **Network failure:** route failure, channel liquidity exhaustion, invoice expiration, or transport disconnect.

11.2 What SATS-402 solves

- preimage-bound delivery of encrypted responses;
- no facilitator custody in core modes;
- no facilitator credit exposure in same-hash bridge mode;
- CLTV-safe bridge acceptance;
- retry/idempotency for paid responses;
- receipt-level evidence of payment and delivery;
- bounded session and streaming authorization.

11.3 What SATS-402 does not solve

- semantic truth of the delivered payload;
- provider fraud in high-value transactions without contracts or reputation;
- regulatory classification for every jurisdiction and business model;
- public Lightning liquidity quality for every path;
- wallet UX for wallets that do not expose preimages to application code.

Oracle boundary

SATS-402 can prove that a ciphertext was bound to a payment and became decryptable by the agent. It cannot prove that the decrypted content is semantically correct, useful, or fair-priced. That remains a merchant reputation, schema validation, refund, or contract-layer problem.

12. Reliability Layer

12.1 Path cascade

- direct or private channel route;

- gateway relay with same-hash bridge;
- public Lightning routing;
- LSP-assisted channel/liquidity preparation;
- opt-in optimistic fast path for ultra-low-value trusted sessions only.

The core bridge should not use the optimistic path. It exists only as a capped UX accelerator after reputation, exposure limits, and kill switches are in place.

12.2 Merchant risk scoring

Merchants should be scored by late reveal rate, average reveal delay, failed settlement rate, excessive CLTV settings, unresolved HTLC count, dispute rate, and payload schema failure rate. Risky merchants can be restricted to atomic_response mode, direct channels, lower in-flight quotas, or manual review.

13. Integration With AI Infrastructure

13.1 Firecrawl-style web-data delivery

Firecrawl's MCP server exposes web scraping, search, page interaction, deep research, browser sessions, cloud/self-hosted support, and streamable HTTP support.[10] A SATS-402 wrapper demonstrates that AI web-data responses can be encrypted and delivered under Bitcoin-native payment settlement without bypassing provider authentication, billing, API keys, or rate limits.

13.2 Apify-style Actor result delivery

Apify's MCP server lets agents use Actors for social media data, maps, e-commerce, search results, and automation. The Apify MCP documentation also describes agentic payments through x402 and Skyfire; x402 pays with USDC on Base, while Skyfire uses PAY tokens.[11] A SATS-402 wrapper demonstrates the complementary Bitcoin-native path for Actor result delivery.

13.3 Public proof artifacts

Artifact	Purpose	Link
Core real-LND proof	Local real-LND bridge demonstration using regtest.	npm run demo:real
Firecrawl wrapper	Permissionless AI web-data wrapper.	https://github.com/Lumen-Founder/firecrawl-sats402-wrapper
Firecrawl draft PR	Optional upstream example proposal.	https://github.com/firecrawl/firecrawl-mcp-server/pull/221
Apify wrapper	Permissionless Actor-result wrapper.	https://github.com/Lumen-Founder/apify-sats402-wrapper
Apify draft PR	Optional upstream example proposal.	https://github.com/apify/apify-mcp-server/pull/757

14. Implementation Evidence

The current prototype has executed the following local proof path:

- boot local regtest bitcoind;
- boot three LND nodes: Agent, Gateway, Merchant;
- initialize and unlock wallets;
- mine regtest funds;
- open Agent-to-Gateway and Gateway-to-Merchant channels;
- create a real gateway hold invoice;
- pay a real merchant invoice;
- observe the real Lightning preimage;
- settle the agent-side hold invoice with the same preimage;
- decrypt the API response locally;
- issue a receipt showing custody: false and credit_extended: false.

The LND APIs used in the proof align with documented primitives: AddInvoice creates invoices with unique payment preimages, AddHoldInvoice ties a hold invoice to a supplied hash, and SettleInvoice settles an accepted invoice.[7,6,8]

15. Commercial Positioning

15.1 Initial wedge

The initial wedge is not all payments. It is paid delivery of API and MCP responses for AI agents:

- web-data extraction;
- search and research context;
- Actor results;
- small inference outputs;
- premium MCP tools;
- browser-session milestones.

15.2 Why this matters for Bitcoin

If x402, AP2, and MCP become the default path for agentic commerce but Bitcoin is absent from settlement and delivery, the next machine-to-machine economy may compound around non-Bitcoin rails. SATS-402 positions Lightning as more than a payment network: it becomes a cryptographic delivery substrate for software resources.

15.3 Why the permissionless wrappers matter

Firecrawl and Apify are not final customers by default. They are proof surfaces. They show that SATS-402 can be inserted into pure AI infrastructure and Actor-based automation without waiting for formal partnerships. The point is not to bypass provider economics. The point is to prove that Bitcoin-native paid delivery can wrap real AI-infrastructure outputs while respecting provider authentication and billing.

16. Roadmap

Version	Name	Milestone
v0.4	Atomic Bridge	Real-LND same-hash bridge, CLTV safety, local response decryption, receipts.
v0.5	Session Ticket	One payment/ticket for bounded sessions, budget caveats, expiry, agent binding.
v0.6	Epoch Streaming	One Lightning preimage per epoch, local chunk key derivation, SSE/WebSocket/gRPC frames.
v0.7	Mandated Budget Delegation	AP2-style mandates, policy engine, human approval thresholds, audit trail.
v0.8	Reliability Layer	merchant risk scores, liquidity manager, route analytics, resume/replay recovery.
v1.0	Production Pilot	25-50 paid API/MCP endpoints, real merchant pilots, security review, legal review.

17. Open Questions

17.1 Wallet preimage exposure

Atomic delivery requires the agent runtime or wallet adapter to access payment preimages. LND, CLN, and LDK-backed wallets can support this, but consumer-hosted wallets may not expose preimages cleanly. SATS-402 should therefore begin with developer/agent infrastructure, not retail wallets.

17.2 Production Lightning liquidity

The local proof demonstrates the primitive, not universal public-route reliability. Production requires LSP integration, routing telemetry, merchant direct channels, quotas, and careful CLTV policies.

17.3 Regulatory posture

The protocol is designed to avoid custody and stored-value balances in its core mode. However, production deployment still requires legal analysis around money transmission, payment facilitation, settlement guarantees, and jurisdiction-specific obligations.

17.4 Semantic disputes

Schema validation and delivery receipts are not semantic truth. A production system should include merchant terms, refund policies, reputation, enterprise SLAs, and optional arbitration for higher-value use cases.

18. Conclusion

SATS-402 is a Bitcoin-native protocol for agentic paid delivery. Its core insight is that a Lightning preimage can be more than payment proof. When combined with ECDH, HKDF, response encryption, CLTV-safe same-hash bridging, and signed receipts, it becomes a delivery root for machine-to-machine commerce.

The protocol family deliberately avoids overextension: Atomic Response for discrete paid resources; Same-Hash HTLC Bridge for non-custodial facilitated settlement; Session Ticket for long-running access; Chunked Streaming Settlement for streams without HTLC-per-chunk overhead; AP2-style mandates for authorization; L402-style preimage-backed credentials for access continuity; and x402-style negotiation and facilitator ergonomics for developer adoption.

Final claim

SATS-402 does not force every streaming byte through Lightning. It uses Lightning for settlement epochs, L402-style tickets for session continuity, AP2-style mandates for budget authority, and preimage-derived local keys for paid delivery.

Appendix A: Minimal Atomic Bridge Pseudocode

```
function atomicBridge(request):
  envelope = parseSATS402Request(request)
  payload = merchant.buildResponse(request)

  S = random32()
  H = sha256(S)

  merchantInvoice = merchant.lnd.addInvoice({ preimage: S, amount })
  encrypted = encrypt(payload, key = HKDF(S || ECDH(agent, merchant)))

  route = gateway.lnd.queryRoute(merchant, amount)
  cltv = computeCLTV(route)
  assert incomingExpiry >= outgoingFirstHopExpiry + bridgeSafetyDelta

  holdInvoice = gateway.lnd.addHoldInvoice({ hash: H, amount, cltv })

  agentPayment = agent.lnd.sendPayment(holdInvoice)
  waitUntilAccepted(holdInvoice)

  outgoing = gateway.lnd.sendPayment(merchantInvoice)
  S_observed = outgoing.preimage

  gateway.lnd.settleInvoice(S_observed)
  agentPreimage = agentPayment.preimage

  plaintext = decrypt(encrypted, HKDF(agentPreimage || ECDH(agent, merchant)))
  return receipt(plaintext, payment, delivery)
```

Appendix B: Production Readiness Checklist

Area	Required before production
Security	External review of key derivation, AEAD usage, receipt signatures, replay handling.
Lightning	LSP strategy, route scoring, liquidity manager, CLTV policy, merchant quotas.
Wallets	Preimage-aware wallet adapters for LND, CLN, LDK, and agent wallet providers.
Compliance	Counsel memo on custody, money transmission, settlement guarantees, and stored value.
Operational reliability	Monitoring for route failures, late reveal, invoice expiry, duplicate payments, cache corruption.
Developer UX	SDK wrappers for Node, Python, FastAPI, Express, Cloudflare Workers, MCP clients.
Market proof	3-5 paid API/MCP pilots with repeat agent usage and signed customer quotes.

Appendix C: Glossary

Term	Definition
Agent	Software actor that requests and pays for resources.
Atomic Response	SATS-402 mode for one paid response encrypted under a preimage-derived key.
CLTV	CheckLockTimeVerify; block-height based expiry used in Lightning HTLCs.
Epoch	A settlement interval for streaming; one Lightning preimage unlocks many chunks.
Facilitator	Service that verifies/coordinates payment and delivery without custody in core mode.
HTLC	Hash Time Locked Contract; Lightning conditional payment construction.
L402	Lightning HTTP 402 credential model using Macaroons and payment preimages.
MCP	Model Context Protocol, used by agents to discover and call tools.
Preimage	Secret S whose hash H is the Lightning payment hash.

Term	Definition
Same-Hash Bridge	SATS-402 bridge where incoming and outgoing invoices share the same payment hash.
Session Ticket	Bounded authorization/access artifact for repeated calls or long sessions.

References

1. x402 Documentation, "Facilitator." https://docs.x402.org/core-concepts/facilitator. Accessed April 27, 2026.

2. x402 Documentation, "Payment-Identifier (Idempotency)." https://docs.x402.org/extensions/payment-identifier. Accessed April 27, 2026.

3. Lightning Labs Builder's Guide, "L402." https://docs.lightning.engineering/the-lightning-network/l402/l402. Accessed April 27, 2026.

4. Lightning Labs Builder's Guide, "L402: Lightning HTTP 402 Protocol." https://docs.lightning.engineering/the-lightning-network/l402. Accessed April 27, 2026.

5. Google Cloud Blog, "Powering AI commerce with the new Agent Payments Protocol (AP2)." https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol. Accessed April 27, 2026.

6. Lightning Labs API Reference, "AddHoldInvoice." https://lightning.engineering/api-docs/api/lnd/invoices/add-hold-invoice/. Accessed April 27, 2026.

7. Lightning Labs API Reference, "AddInvoice." https://lightning.engineering/api-docs/api/lnd/lightning/add-invoice/. Accessed April 27, 2026.

8. Lightning Labs API Reference, "SettleInvoice." https://lightning.engineering/api-docs/api/lnd/invoices/settle-invoice/. Accessed April 27, 2026.

9. Lightning Network BOLTs, "BOLT #2: Peer Protocol for Channel Management." https://github.com/lightning/bolts/blob/master/02-peer-protocol.md. Accessed April 27, 2026.

10. Firecrawl Documentation, "Firecrawl MCP Server." https://docs.firecrawl.dev/mcp-server. Accessed April 27, 2026.

11. Apify GitHub, "apify/apify-mcp-server." https://github.com/apify/apify-mcp-server. Accessed April 27, 2026.

Prepared for technical diligence. This document describes a protocol proposal and prototype evidence, not a representation of production readiness or regulatory status.